Backend Development With Node.js

Project documentation

1. Introduction :

- Project Title: Micro Goal
- Team ID: SWTID-2026-4832
- Team leader: Meena K
- Id:24bsccs31@cttewc.edu.in
- Team Members:
- Santhiya S
- Id:24bsccs05@cttewc.edu.in
- Alexandra B
- Id:24bsccs20@cttewc.edu.in
- Keerthika K
- Id :24bsccs52@cttewc.edu.in

2. Project Overview:

MicroGoals is a backend REST API developed using Node.js, Express.js, and MongoDB. The system allows users to register, log in, and manage their personal goals. The application uses JWT authentication to ensure that only authorized users can access protected routes. Users can create, read, update, and delete their goals securely. The project follows MVC architecture, which helps in organizing the code efficiently and making the system scalable.

3. Objectives of the Project:

To build a secure backend system for goal management.
To implement user authentication using JWT.
To perform CRUD operations on goals.
To ensure data security using middleware.
To learn REST API development using Node.js.

4. Technologies Used:

Node.js – Used as the runtime environment for executing JavaScript on the server side.
Express.js – A web framework used to build REST APIs easily.
MongoDB – NoSQL database used to store user and goal data.
JWT (JSON Web Token) – Used for authentication and secure API access.
Thunder Client – Used to test API endpoints.

5. System Architecture:

- The MicroGoals backend follows MVC architecture:
- Model: Defines database schemas (User and Goal).
- View: Not used because it is only a backend API.Controller/Routes: Handles API logic and user requests.

Flow:

- User → API Request → Middleware → Controller → Database → Response.

6. Features
- Write the main features.
- User Registration
- Secure Login with JWT
- Create Goal
- Update Goal
- Delete Goal
- View Goals
- Protected Routes
- Centralized Error Handling

7. Project Setup
- Install Node.js and MongoDB
- Create project
- npm init -y
- Install dependencies:
- npm install express mongoose jsonwebtoken bcryptjs dotenv

Start server:
- node server.js

8. API Development
- Explain APIs briefly.
- Authentication APIs
- User Register
- User Login
- JWT Token Generation
- Goal APIs
- Create Goal
- Get Goal
- Update Goal
- Delete Goal
- All routes are protected using Auth Middleware.

9. Middleware
- Explain middleware.
- Auth Middleware
- Verifies JWT token before allowing access to protected routes.
- Error Middleware
- Handles application errors and returns proper status codes like:
- 200 – Success
- 201 – Created
- 400 – Bad Request
- 401 – Unauthorized
- 404 – Not Found

10. Database Development:

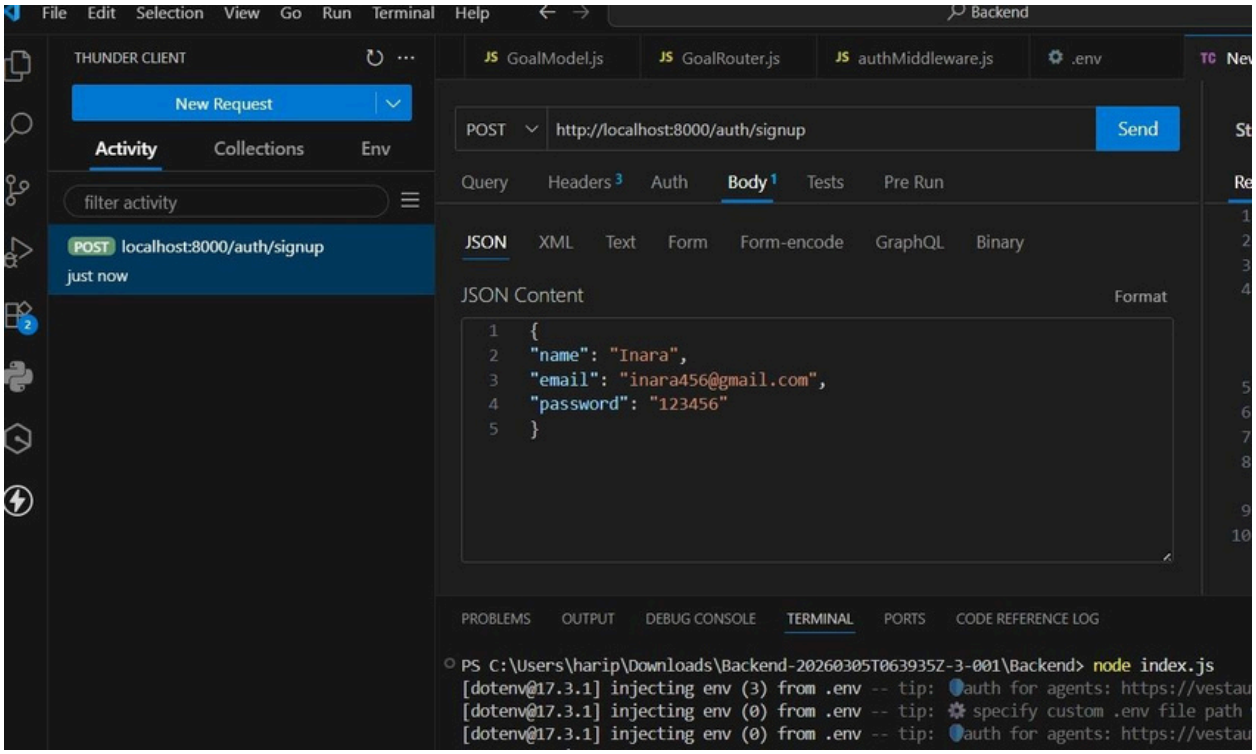
- MongoDB is used to store user and goal data. The. connection string is stored in the .env file and connectedusing mongoose.connect() in the main server file.

11.Project Execution

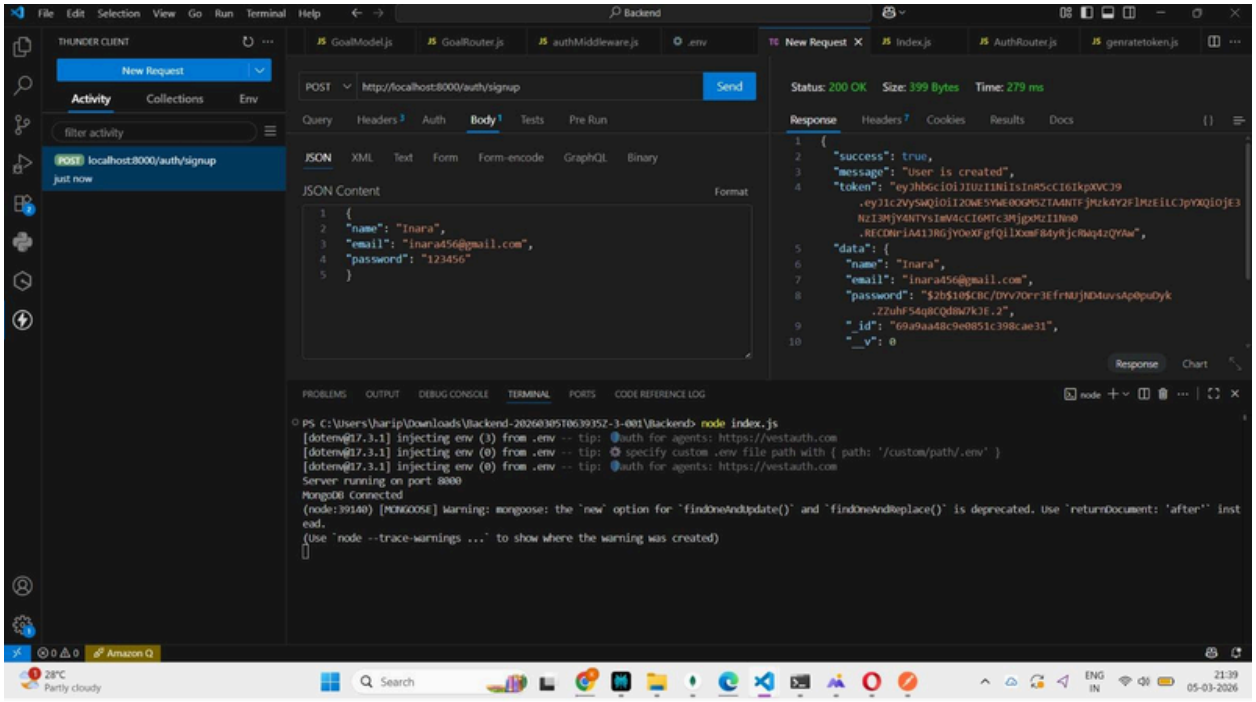
Steps:

- Start MongoDB
- Run Node server
- Test APIs in Thunder Client
- Register User
- Login and get JWT token
- Use token for CRUD operations

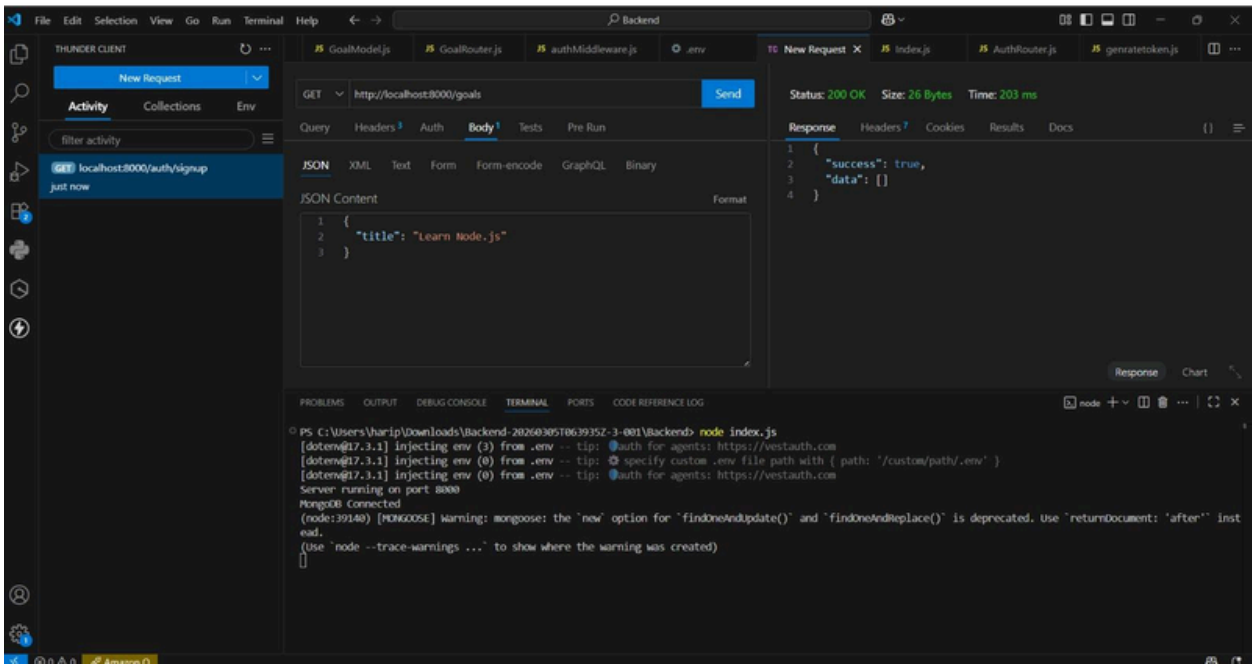
12.Output Screenshots:



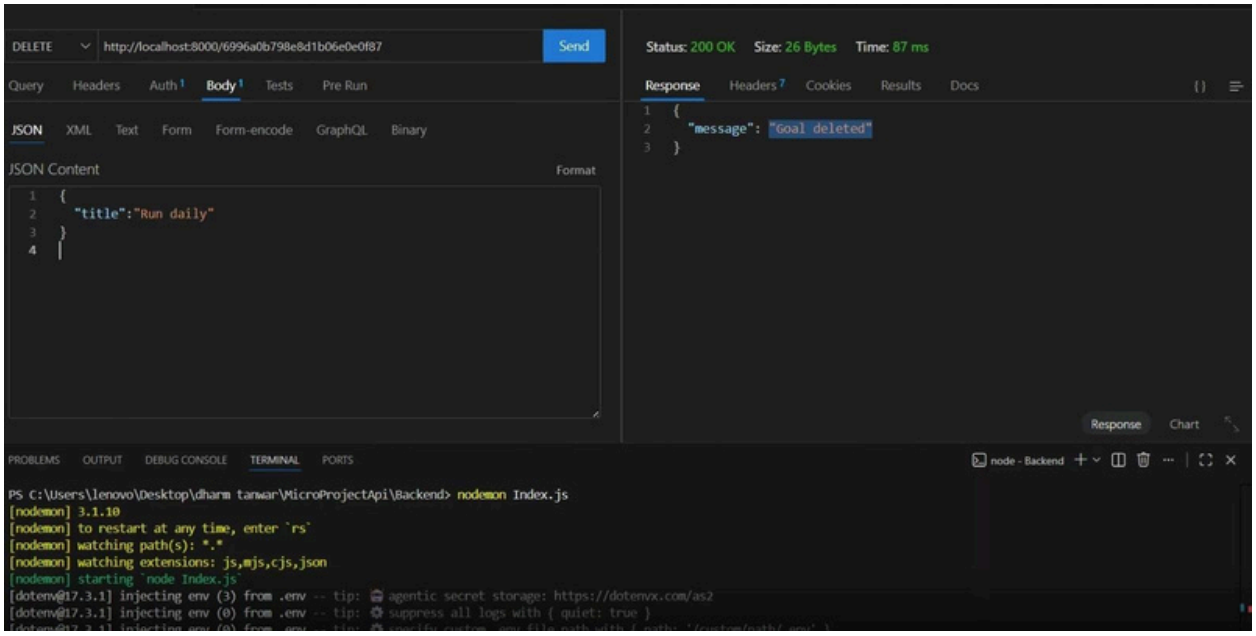
Phase 2 :


File Edit Selection View Go Run Terminal Help
Backend
THUNDER CLIENT
New Request
Activity Collections Env
filter activity
POST localhost:8000/auth/signup
just now
GoalModel.js
GoalRouter.js
authMiddleware.js
.env
New Request
Index.js
AuthRouter.js
genratetoken.js
POST http://localhost:8000/auth/signup
Send
Query Headers 3 Auth Body 1 Tests Pre Run
JSON XML Text Form Form-encode GraphQL Binary
JSON Content
Format
{
"name": "Inara",
"email": "inara456@gmail.com",
"password": "123456"
}
Status: 200 OK Size: 399 Bytes Time: 279 ms
Response Headers 7 Cookies Results Docs
{
"success": true,
"message": "User is created",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJ1c2VySWQiOiI2OWE5YWE0OGM5ZTA4NTFjMzk4Y2FlMzEiLCJpYXQiOjE3
NzI3MjY4NTYsIm4cCI6MTc3MjgxMzI1Nn0
.RECDNriA41JRGjYOxXFgfQilXomF84yRjcRWq4zQYAw",
"data": {
"name": "Inara",
"email": "inara456@gmail.com",
"password": "$2b$10$CBC/DYv7Orr3EfrNUjND4uvsAp0puDyk
.ZZuhF54q8CQd8W7kJE.2",
"_id": "69a9aa48c9e0851c398cae31",
"__v": 0
Response Chart
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
PS C:\Users\harip\Downloads\Backend-20260305T063935Z-3-001\Backend> node index.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: auth for agents: https://vestauth.com
[dotenv@17.3.1] injecting env (0) from .env -- tip: specify custom .env file path with { path: '/custom/path/.env' }
[dotenv@17.3.1] injecting env (0) from .env -- tip: auth for agents: https://vestauth.com
Server running on port 8000
MongoDB Connected
(node:39140) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocument: 'after'` inst
ead.
(Use `node --trace-warnings ...` to show where the warning was created)
Amazon Q
28°C Partly cloudy
Search
ENG IN
21:39 05-03-2026

Phase 3:


File Edit Selection View Go Run Terminal Help
Backend
THUNDER CLIENT
New Request
Activity Collections Env
filter activity
GET localhost:8000/auth/signup
just now
GoalModel.js
GoalRouter.js
authMiddleware.js
.env
New Request
Index.js
AuthRouter.js
genratetoken.js
GET http://localhost:8000/goals
Send
Query Headers 3 Auth Body 1 Tests Pre Run
JSON XML Text Form Form-encode GraphQL Binary
JSON Content
Format
{
"title": "Learn Node.js"
}
Status: 200 OK Size: 26 Bytes Time: 203 ms
Response Headers 7 Cookies Results Docs
{
"success": true,
"data": []
}
Response Chart
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
PS C:\Users\harip\Downloads\Backend-20260305T063935Z-3-001\Backend> node index.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: auth for agents: https://vestauth.com
[dotenv@17.3.1] injecting env (0) from .env -- tip: specify custom .env file path with { path: '/custom/path/.env' }
[dotenv@17.3.1] injecting env (0) from .env -- tip: auth for agents: https://vestauth.com
Server running on port 8000
MongoDB Connected
(node:39140) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocument: 'after'` inst
ead.
(Use `node --trace-warnings ...` to show where the warning was created)
Amazon Q

Phase 4:



Demo Video

- https://drive.google.com/file/d/1Y3NncYJe_EI3I5gXD4SLwYLLMvwCUfwp/view?usp=drivesdk

- The demo video shows the complete working of the MicroGoals API including:
- User registration
- Login process
- Token generation
- CRUD operations
- Error handling